

Essential attributes of good software

Good software means software that works properly, is safe, and is easy to use. There are four essential attributes of good software.

Unit-1_chapter 1

1. Maintainability

Maintainability means the software should be easy to change and update.

In real life, user needs change again and again. So software must be easy to modify. If software is difficult to change, it will create problems.

Example:

College management software may need new features like online fees. It should be easy to add.

Easy keyword to remember:

👉 Easy to change

2. Dependability and Security

Dependability means software should work correctly without failure.

Security means software should protect data from unauthorized users.

Software should not give wrong results. It should also keep user data safe.

Example:

ATM software should give correct money and keep PIN safe.

Easy keyword to remember:

👉 Correct and Safe

3. Efficiency

Efficiency means software should use system resources properly.

Software should not waste memory, CPU, or time. It should work fast and smoothly.

If software is not efficient, system will become slow.

Example:

Mobile apps should not hang or slow down phone.

Easy keyword to remember:

👉 Fast and not waste resources

4. Acceptability

Acceptability means software should be easy to use and easy to understand.

Users should feel comfortable while using software. It should also work with other systems.

Example:

Website should be simple and user-friendly.

Easy keyword to remember:

👉 Easy to use

Software Engineering Ethics – Definition and Explanation

Definition

Software engineering ethics means the moral rules and responsibilities that software engineers must follow while developing software.

It tells software engineers to work honestly, safely, and responsibly.

Explanation in detail

Software engineers develop software that people use in daily life like banking, hospital, railway, etc. So, they have a big responsibility.

They must follow ethical principles to protect users, company, and society.

Ethics means doing the right thing.

Need for Software Engineering Ethics

Software is used in important systems like:

- Banking
- Hospitals
- Railway
- Air traffic control

If software fails, it can cause serious damage. So engineers must follow ethical rules.

Main principles of software engineering ethics

1. Confidentiality

Software engineers must keep secret information safe.

They should not share client or company data with others.

Example:

Bank software engineer should not share customer details.

2. Competence

Engineers should work only in areas they understand.

They should not do work they don't know properly.

Example:

If engineer does not know cybersecurity, they should not design security system.

3. Intellectual Property Rights

Engineers should respect copyrights, patents, and licenses.

They should not copy others' software illegally.

Example:

Do not copy someone else's program and say it is yours.

4. Computer Misuse

Engineers should not misuse computers.

They should not hack systems or create viruses.

Example:

Engineer should not hack company database.

Why software engineering ethics is important

- To protect user data
- To avoid misuse of software
- To develop safe software
- To build trust
- To protect society

Example

If engineer finds a software error that can harm users, they must report it, not hide it.

Conclusion

Software engineering ethics is very important. It ensures software engineers work honestly, safely, and responsibly. It protects users, company, and society.

Short exam definition (2 marks)

Software engineering ethics is the set of moral principles that software engineers must follow while developing software.

Outline ACM / IEEE Code of Ethics

The **ACM/IEEE Code of Ethics** is a set of **8 principles that software engineers must follow to behave professionally and ethically**. It helps engineers work honestly and protect society.

SEPM_Answers

8 Principles of ACM / IEEE Code of Ethics

1. Public

Software engineers should act in the **public interest**.

They should develop software that is safe and does not harm people.

2. Client and Employer

Engineers should work in the **best interest of their client and employer**.

They should be honest and loyal.

3. Product

Engineers should ensure the **software product meets high quality standards**.

Software should be reliable and safe.

4. Judgment

Engineers should maintain **honesty and independence in their professional judgment**.

They should not give wrong decisions.

5. Management

Software managers should follow and promote **ethical management**.

They should manage projects properly.

6. Profession

Engineers should maintain the **respect and reputation of the profession**.

They should behave professionally.

7. Colleagues

Engineers should be **fair and supportive to their colleagues**.

They should respect teamwork.

8. Self

Engineers should **improve their knowledge and skills continuously**.

They should follow ethical practices in their career.

Conclusion

The ACM/IEEE Code of Ethics provides guidelines for software engineers to work honestly, responsibly, and professionally.

Short Answer (4 marks)

The ACM/IEEE Code of Ethics consists of 8 principles:

1. Public
2. Client and Employer
3. Product
4. Judgment
5. Management
6. Profession
7. Colleagues
8. Self

Memory Trick (Very useful for exam)

P C P J M P C S

Public

Client

Product

Judgment

Management

Profession

Colleagues
Self

Insulin Pump Control System

What it is

The insulin pump control system is an embedded software system used by diabetic patients. It automatically monitors blood sugar level and delivers the correct amount of insulin to the patient.

It works like an artificial pancreas and helps patients maintain normal sugar level.

SEPM_Answers

How it works

First, a micro-sensor placed in the patient measures the blood sugar level. This information is sent to the controller.

The controller calculates how much insulin is needed.

Then the controller sends signals to the pump.

Finally, the pump delivers the correct amount of insulin into the patient body.

Key concerns

1. Safety:

It is a safety-critical system. Wrong insulin dose may cause coma or death.

2. Reliability:

The system must work continuously without failure.

3. Real-time response:

The system must respond immediately to sugar level changes.

Conclusion

Thus, the insulin pump system helps diabetic patients by automatically delivering insulin safely.

2. Wilderness Weather Station

What it is

The wilderness weather station is a data collection system used in remote areas.

It collects weather information like temperature, wind speed, pressure, and rainfall.

How it works

Sensors collect weather data from the environment.

The weather station system processes the data.

Then the data is transmitted to the central data management system.

The maintenance system monitors the weather station and detects faults.

Key concerns

1. Dependability:

The system must work properly in remote areas where repair is difficult.

2. Maintainability:

The system should allow remote maintenance and software updates.

3. Power management:

The system uses battery power, so power must be managed carefully.

4. Autonomous operation:

The system must work without human control.

Conclusion

Thus, the weather station system helps in collecting and managing weather information.

3. Mental Health Care Patient Management System (MHC-PMS)

What it is

Mental Health Care Patient Management System is an information system used to store and manage patient records.

It helps doctors and hospitals to provide proper treatment.

How it works

Doctors enter patient details into the system.

The system stores the data in a centralized database.

Doctors can view patient history and treatment details.

The system generates reports and gives warnings if any risk is detected.

Key concerns

1. Privacy:

Patient information must be kept confidential.

2. Safety:

Wrong information may cause wrong treatment.

3. Security:

Unauthorized persons must not access patient data.

4. Availability:

The system must be available when needed.

Conclusion

Thus, the MHC-PMS system helps in managing patient information and improves healthcare services.

Waterfall Model in Software Engineering

Definition

The Waterfall model is a software development process where work is done step-by-step in sequence, and each phase must be completed before moving to the next phase.

SEPM_Answers

It is called waterfall because the process flows downward like a waterfall.

Phases of Waterfall Model

There are 5 main phases:

1. Requirements Analysis

- In this phase, requirements are collected from the customer.
- The system goals and functions are defined.

Example:

What the software should do is decided.

2. System Design

- In this phase, system design is prepared.
- Architecture and structure of software are created.

Example:

How the software will work is decided.

3. Implementation and Unit Testing

- In this phase, programmers write the code.
- Each module is tested separately.

Example:

Program is developed.

4. Integration and System Testing

- All modules are combined.
- The complete system is tested.

Example:

Check whether software works correctly.

5. Operation and Maintenance

- Software is delivered to customer.
- Errors are fixed and updates are done.

Example:

Software is used and maintained.

Characteristics of Waterfall Model

- Simple and easy to understand
- Each phase is completed before next phase
- No overlapping between phases

Advantages

1. Easy to understand
2. Easy to manage
3. Proper documentation
4. Works well when requirements are clear

Disadvantages

1. Difficult to change requirements
2. Errors found late
3. Not suitable for complex projects

Conclusion (Exam ending)

Waterfall model is a simple and sequential software development model where each phase is completed before moving to the next phase.

Incremental Model in Software Engineering

Definition

The Incremental model is a software development model where the software is developed and delivered in small parts called increments.

Each increment adds new features to the system until the full software is completed.

SEPM_Answers

How it works

In this model, software is not developed at once.

Instead, it is developed step by step.

- First, basic features are developed and delivered.
- Then, more features are added in the next increment.
- This process continues until the full system is completed.

Each increment includes:

- Requirements
- Design
- Implementation
- Testing

Example

Bank software:

- Increment 1 → Login feature
- Increment 2 → Money deposit
- Increment 3 → Money transfer

Phases in Incremental Model

Each increment has these phases:

1. Requirements analysis
2. Design
3. Implementation
4. Testing

5. Delivery

Advantages

1. Easy to add new features
2. Customer can see software early
3. Errors can be found early
4. Less risk

Disadvantages

1. Needs good planning
2. System design may become complex
3. Not suitable for small projects

Conclusion (Exam ending)

Incremental model develops software step by step, and each increment adds new features until the complete system is developed.

Short 4-mark exam answer

Incremental model is a software development model where software is developed and delivered in small parts. Each increment adds new features until the system is complete.

Functional and Non-Functional Requirements

Definition

Requirements are the description of what the software should do and the constraints under which it should operate.

Copy of Ch4

There are two types:

1. Functional requirements
2. Non-functional requirements

1. Functional Requirements

Definition

Functional requirements are the services and functions that the system should provide. They describe what the system does.

Copy of Ch4

Explanation

These requirements tell:

- What system should do
- How system should respond
- What output system gives

They depend on:

- Type of software
- User needs

Example

In ATM system:

- System should withdraw money
- System should show balance
- System should accept PIN

In MHC-PMS system:

- System should store patient data

- System should generate patient report
- Copy of Ch4

Key point to remember

Functional requirement = What system does

2. Non-Functional Requirements

Definition

Non-functional requirements are the constraints and properties of the system.

They describe how the system works.

Copy of Ch4

Explanation

These requirements define:

- Performance
- Security
- Reliability
- Speed

They apply to the whole system.

Examples

ATM system:

- System should respond in 2 seconds
- System should be secure
- System should work 24 hours

MHC-PMS system:

- System should keep patient data private
- System should be reliable

Types of Non-Functional Requirements

1. Product requirement

Example: speed, reliability

2. Organizational requirement

Example: programming language used

3. External requirement

Example: legal and security requirements

Copy of Ch4

Key point to remember

Non-functional requirement = How system works

Difference between Functional and Non-Functional Requirements

Functional	Non-Functional
What system does	How system works
Defines functions	Defines quality
Example: Withdraw money	Example: Fast response
Service requirement	Constraint requirement

Conclusion

Functional requirements define system functions.

Non-functional requirements define system quality and constraints.

Both are important for software development.

Memory Trick (Very important)

Remember:

Functional → WHAT

Non-functional → HOW

Software Process Activities

Definition

Software process activities are the main steps performed to develop and maintain a software system.

These activities help to define, design, test, and update software.

Copy of Ch1

There are four main process activities:

1. Software Specification
2. Software Development
3. Software Validation
4. Software Evolution

1. Software Specification

Meaning

Software specification is the process of defining what the system should do and its requirements.

Copy of Ch2

It includes:

- Identifying customer needs
- Writing requirements

Example

ATM system should:

- Withdraw money
- Show balance

2. Software Development

Meaning

Software development is the process of designing and coding the software.

Copy of Ch2

It includes:

- Software design
- Programming

Example

Writing program for ATM machine.

3. Software Validation

Meaning

Software validation is the process of checking whether software works correctly and meets customer requirements.

Copy of Ch2

It includes:

- Testing
- Checking errors

Example

Testing ATM gives correct money.

4. Software Evolution

Meaning

Software evolution is the process of modifying software after delivery.

Copy of Ch2

It includes:

- Adding new features
- Fixing errors

Example

Adding new feature in banking app.

Conclusion

Software process activities are:

- Specification → define requirements
- Development → design and coding
- Validation → testing
- Evolution → updating software

These activities are important for software development.

Short Answer (4 marks)

Software process activities are the main activities in software development:

1. Specification
2. Development
3. Validation
4. Evolution

Memory Trick

Remember:

S D V E

Specification

Development

Validation

Evolution